

Ek line me yaad rakho:

- **WHERE** = Individual rows ko filter karta hai.
- **GROUP BY** = Rows ko groups me convert karta hai.
- **HAVING** = Ban chuke groups ko filter karta hai.

2. A-Block departments ke students

SQL

```
SELECT *  
FROM student  
WHERE dept_id IN  
(  
    SELECT dept_id  
    FROM department  
    WHERE block_name='A-Block'  
);
```

3. Dr. Sharma ke department ke students

SQL

```
SELECT *  
FROM student  
WHERE dept_id =  
(  
    SELECT dept_id  
    FROM department  
    WHERE hod_name='Dr. Sharma'  
);
```

4. Electrical department ke students

SQL

```
SELECT *  
FROM student  
WHERE dept_id=  
(  
SELECT dept_id  
FROM department  
WHERE dept_name='Electrical'  
);
```

5. Maximum marks wale student

SQL

```
SELECT *  
FROM student  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
);
```

6. Department-wise topper (Correlated Subquery)

SQL

```
SELECT *  
FROM student s  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

7. Department average se jyada marks

SQL

```
SELECT *  
FROM student s  
WHERE marks>  
(  
SELECT AVG(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

8. Department average se kam marks

SQL

```
SELECT *  
FROM student s  
WHERE marks <  
(  
SELECT AVG(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

9. Same city as Tarun

SQL

```
SELECT *  
FROM student  
WHERE city=  
(  
  SELECT city  
  FROM student  
  WHERE name='Tarun'  
);
```

10. Same department as Tarun

SQL

```
SELECT *  
FROM student  
WHERE dept_id=  
(  
  SELECT dept_id  
  FROM student  
  WHERE name='Tarun'  
);
```

11. Highest marks in IT Department

SQL

```
SELECT *
FROM student
WHERE dept_id=
(
SELECT dept_id
FROM department
WHERE dept_name='IT'
)
AND marks=
(
SELECT MAX(marks)
FROM student
WHERE dept_id=
(
SELECT dept_id
FROM department
WHERE dept_name='IT'
)
);
```

13. Students whose marks are greater than ALL students of Electrical

SQL

```
SELECT *
FROM student
WHERE marks >
ALL
(
  SELECT marks
  FROM student
  WHERE dept_id =
  (
    SELECT dept_id
    FROM department
    WHERE dept_name = 'Electrical'
  )
);
```

14. Students whose marks are greater than ANY student of Electrical

SQL

```
SELECT *
FROM student
WHERE marks >
ANY
(
  SELECT marks
  FROM student
  WHERE dept_id =
  (
    SELECT dept_id
    FROM department
    WHERE dept_name = 'Electrical'
  )
);
```

15. Students NOT in Mechanical department

SQL

```
SELECT *  
FROM student  
WHERE dept_id<>  
(  
SELECT dept_id  
FROM department  
WHERE dept_name='Mechanical'  
);
```


16. EXISTS

Departments having at least one student.

SQL

```
SELECT *  
FROM department d  
WHERE EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
);
```

17. NOT EXISTS

Departments having no students.

SQL

```
SELECT *  
FROM department d  
WHERE NOT EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
);
```

18. Students from the department having the maximum number of students

SQL

```
SELECT *  
FROM student  
WHERE dept_id=  
(  
  SELECT dept_id  
  FROM student  
  GROUP BY dept_id  
  ORDER BY COUNT(*) DESC  
  LIMIT 1  
);
```



20. Students whose department average is above overall average

```
</> SQL

SELECT *
FROM student
WHERE dept_id IN
(
SELECT dept_id
FROM student
GROUP BY dept_id
HAVING AVG(marks)>
(
SELECT AVG(marks)
FROM student
)
);
```

High-Level JOIN + Foreign Key Query

Department name ke saath student details:

```
</> SQL

SELECT s.id,
       s.name,
       s.marks,
       d.dept_name,
       d.hod_name
FROM student s
INNER JOIN department d
ON s.dept_id=d.dept_id;
```

Topper of Every Department

`</>` SQL

```
SELECT s.name,  
       s.marks,  
       d.dept_name  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
WHERE s.marks=  
(  
  SELECT MAX(marks)  
  FROM student  
  WHERE dept_id=s.dept_id  
);
```

Department-wise Average > Overall Average

SQL

```
SELECT d.dept_name,  
       AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name  
HAVING AVG(s.marks)>  
(  
SELECT AVG(marks)  
FROM student  
);
```

1. Department-wise Student Count

SQL

```
SELECT dept_id,  
COUNT(*) AS total_students  
FROM student  
GROUP BY dept_id;
```

2. City-wise Student Count

SQL

```
SELECT city,  
COUNT(*) AS total_students  
FROM student  
GROUP BY city;
```



3. Gender-wise Student Count

SQL

```
SELECT gender,  
COUNT(*) AS total_students  
FROM student  
GROUP BY gender;
```

4. Department-wise Average Marks

SQL

```
SELECT dept_id,  
AVG(marks) AS average_marks  
FROM student  
GROUP BY dept_id;
```



5. Department-wise Maximum Marks

`</>` SQL

```
SELECT dept_id,  
MAX(marks) AS highest_marks  
FROM student  
GROUP BY dept_id;
```

6. Department-wise Minimum Marks

`</>` SQL

```
SELECT dept_id,  
MIN(marks) AS lowest_marks  
FROM student  
GROUP BY dept_id;
```



7. Department-wise Total Marks

SQL

```
SELECT dept_id,  
SUM(marks) AS total_marks  
FROM student  
GROUP BY dept_id;
```

8. Course Year-wise Average Marks

SQL

```
SELECT course_year,  
AVG(marks)  
FROM student  
GROUP BY course_year;
```

9. City-wise Highest Marks

`</>` SQL

```
SELECT city,  
MAX(marks)  
FROM student  
GROUP BY city;
```

10. City-wise Lowest Marks

`</>` SQL

```
SELECT city,  
MIN(marks)  
FROM student  
GROUP BY city;
```

11. Gender-wise Average Marks

SQL

```
SELECT gender,  
AVG(marks)  
FROM student  
GROUP BY gender;
```

12. Department-wise Average Age

SQL

```
SELECT dept_id,  
AVG(age)  
FROM student  
GROUP BY dept_id;
```



13. City-wise Total Marks

SQL

```
SELECT city,  
SUM(marks)  
FROM student  
GROUP BY city;
```

14. Department-wise Student Count (Department Name ke saath)

SQL

```
SELECT d.dept_name,  
COUNT(*) AS total_students  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name;
```



15. Department-wise Average Marks (Department Name)

SQL

```
SELECT d.dept_name,  
AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name;
```

16. HOD-wise Student Count

SQL

```
SELECT d.hod_name,  
COUNT(*)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.hod_name;
```

18. Block-wise Average Marks

SQL

```
SELECT d.block_name,  
AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.block_name;
```

19. Department + Gender-wise Student Count

SQL

```
SELECT dept_id,  
gender,  
COUNT(*)  
FROM student  
GROUP BY dept_id, gender;
```

20. City + Gender-wise Student Count

SQL

```
SELECT city,  
gender,  
COUNT(*)  
FROM student  
GROUP BY city, gender;
```

21. Department + Course Year-wise Average Marks

SQL

```
SELECT dept_id,  
course_year,  
AVG(marks)  
FROM student  
GROUP BY dept_id, course_year;
```

22. Department-wise Topper Marks

SQL

```
SELECT dept_id,  
MAX(marks) AS topper_marks  
FROM student  
GROUP BY dept_id;
```

23. City-wise Topper Marks

SQL

```
SELECT city,  
MAX(marks)  
FROM student  
GROUP BY city;
```

24. Department-wise Lowest Marks

SQL

```
SELECT d.dept_name,  
MIN(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name;
```

25. Department-wise Total Students and Average Marks

SQL

```
SELECT d.dept_name,  
COUNT(*) AS total_students,  
AVG(s.marks) AS average_marks  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name;
```


GROUP BY + ORDER BY

Highest average marks wale departments

SQL

```
SELECT d.dept_name,  
AVG(s.marks) AS avg_marks  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name  
ORDER BY avg_marks DESC;
```

GROUP BY + HAVING

80 se jyada average marks wale departments

`</>` SQL

```
SELECT d.dept_name,  
       AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name  
HAVING AVG(s.marks)>80;
```

GROUP BY + HAVING + ORDER BY

SQL

```
SELECT d.dept_name,  
COUNT(*) AS total_students,  
AVG(s.marks) AS avg_marks  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name  
HAVING AVG(s.marks)>75  
ORDER BY avg_marks DESC;
```

1. Departments having more than 10 students

SQL

```
SELECT dept_id,  
COUNT(*) AS total_students  
FROM student  
GROUP BY dept_id  
HAVING COUNT(*) > 10;
```

2. Cities having more than 8 students

SQL

```
SELECT city,  
COUNT(*) AS total_students  
FROM student  
GROUP BY city  
HAVING COUNT(*) > 8;
```

...

3. Departments whose average marks > 80

SQL

```
SELECT dept_id,  
AVG(marks) AS avg_marks  
FROM student  
GROUP BY dept_id  
HAVING AVG(marks) > 80;
```

4. Cities whose average marks > 75

SQL

```
SELECT city,  
AVG(marks)  
FROM student  
GROUP BY city  
HAVING AVG(marks) > 75;
```



5. Departments having maximum marks > 90

`</>` SQL

```
SELECT dept_id,  
MAX(marks)  
FROM student  
GROUP BY dept_id  
HAVING MAX(marks) > 90;
```

6. Departments having minimum marks < 60

`</>` SQL

```
SELECT dept_id,  
MIN(marks)  
FROM student  
GROUP BY dept_id  
HAVING MIN(marks) < 60;
```



7. Cities having total marks > 700

SQL

```
SELECT city,  
SUM(marks)  
FROM student  
GROUP BY city  
HAVING SUM(marks) > 700;
```

8. Course years having more than 15 students

SQL

```
SELECT course_year,  
COUNT(*)  
FROM student  
GROUP BY course_year  
HAVING COUNT(*) > 15;
```



9. Gender having average marks > 80

SQL

```
SELECT gender,  
AVG(marks)  
FROM student  
GROUP BY gender  
HAVING AVG(marks) > 80;
```

10. Departments having average age > 20

SQL

```
SELECT dept_id,  
AVG(age)  
FROM student  
GROUP BY dept_id  
HAVING AVG(age) > 20;
```



HAVING + JOIN

11. Department names having more than 10 students

SQL

```
SELECT d.dept_name,  
COUNT(*) AS total_students  
FROM student s  
JOIN department d  
ON s.dept_id = d.dept_id  
GROUP BY d.dept_name  
HAVING COUNT(*) > 10;
```

12. Department names having average marks > 80

SQL

```
SELECT d.dept_name,  
AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id = d.dept_id  
GROUP BY d.dept_name  
HAVING AVG(s.marks) > 80;
```

13. Blocks having more than 20 students

SQL

```
SELECT d.block_name,  
COUNT(*)  
FROM student s  
JOIN department d  
ON s.dept_id = d.dept_id  
GROUP BY d.block_name  
HAVING COUNT(*) > 20;
```

14. HODs whose department average > 80

SQL

```
SELECT d.hod_name,  
AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id = d.dept_id  
GROUP BY d.hod_name  
HAVING AVG(s.marks) > 80;
```



HAVING + WHERE

15. Indore students ka department-wise average > 80

SQL



```
SELECT dept_id,  
AVG(marks)  
FROM student  
WHERE city = 'Indore'  
GROUP BY dept_id  
HAVING AVG(marks) > 80;
```

👉 Pehle `WHERE` sirf Indore ke students lega, phir `GROUP BY` karega, aur uske baad `HAVING` average check

16. Female students ka department-wise count > 5

SQL

```
SELECT dept_id,  
COUNT(*)  
FROM student  
WHERE gender = 'Female'  
GROUP BY dept_id  
HAVING COUNT(*) > 5;
```

HAVING + ORDER BY

17. Highest average marks departments

</> SQL

```
SELECT dept_id,  
AVG(marks) AS avg_marks  
FROM student  
GROUP BY dept_id  
HAVING AVG(marks) > 75  
ORDER BY avg_marks DESC;
```

18. Cities with highest student count

</> SQL

```
SELECT city,  
COUNT(*) AS total  
FROM student  
GROUP BY city  
HAVING COUNT(*) > 5  
ORDER BY total DESC;
```

HAVING + Multiple Conditions

19. Departments having more than 10 students and average > 75

SQL

```
SELECT dept_id,  
COUNT(*) AS total_students,  
AVG(marks) AS avg_marks  
FROM student  
GROUP BY dept_id  
HAVING COUNT(*) > 10  
AND AVG(marks) > 75;
```

20. Cities having average marks between 70 and 85

SQL

```
SELECT city,  
AVG(marks) AS avg_marks  
FROM student  
GROUP BY city  
HAVING AVG(marks) BETWEEN 70 AND 85;
```

Advanced HAVING + Subquery

21. Departments whose average marks are above the overall average

</> SQL

```
SELECT dept_id,  
AVG(marks) AS avg_marks  
FROM student  
GROUP BY dept_id  
HAVING AVG(marks) >  
(  
  SELECT AVG(marks)  
  FROM student  
);
```

22. Departments whose total marks are greater than the average total marks of all departments

</> SQL

```
SELECT dept_id,  
SUM(marks) AS total_marks  
FROM student  
GROUP BY dept_id  
HAVING SUM(marks) >  
(  
  SELECT AVG(total_marks)  
  FROM  
  (  
    SELECT SUM(marks) AS total_marks  
    FROM student  
    GROUP BY dept_id  
  ) AS t  
);
```



2. LEFT JOIN

Definition

Left table ke **saare rows**, right table ke matching rows.

Student ✓✓✓✓✓

Department

Match ✓

No Match → NULL

SQL

```
SELECT s.*,  
       d.dept_name  
FROM student s  
LEFT JOIN department d  
ON s.dept_id=d.dept_id;
```



3. RIGHT JOIN

Right table ke saare rows.

</> SQL

```
SELECT s.name,  
       d.dept_name  
FROM student s  
RIGHT JOIN department d  
ON s.dept_id=d.dept_id;
```


4. FULL OUTER JOIN

MySQL me direct support nahi hai.

UNION se karte hain.

</> SQL

```
SELECT *  
FROM student s  
LEFT JOIN department d  
ON s.dept_id=d.dept_id
```

UNION

```
SELECT *  
FROM student s  
RIGHT JOIN department d  
ON s.dept_id=d.dept_id;
```

5. SELF JOIN

Same table ko khud se join karna.

Example

Same city ke students.

SQL

```
SELECT s1.name,  
       s2.name,  
       s1.city  
FROM student s1  
JOIN student s2  
ON s1.city=s2.city  
AND s1.id<s2.id;
```

Same department

</> SQL

```
SELECT s1.name,  
       s2.name,  
       s1.dept_id  
FROM student s1  
JOIN student s2  
ON s1.dept_id=s2.dept_id  
AND s1.id<s2.id;
```

6. CROSS JOIN

Cartesian Product

65 Students

5 Departments

Result

$65 \times 5 = 325$ Rows

SQL

```
SELECT s.name,  
       d.dept_name  
FROM student s  
CROSS JOIN department d;
```

7. Exclusive LEFT JOIN

Student hai

Department nahi

SQL

```
SELECT s.*  
FROM student s  
LEFT JOIN department d  
ON s.dept_id=d.dept_id  
WHERE d.dept_id IS NULL;
```

8. Exclusive RIGHT JOIN

Department hai

Student nahi

</> SQL

```
SELECT d.*  
FROM student s  
RIGHT JOIN department d  
ON s.dept_id=d.dept_id  
WHERE s.id IS NULL;
```

9. Exclusive FULL JOIN

Sirf unmatched rows

</> SQL

```
SELECT *  
FROM student s  
LEFT JOIN department d  
ON s.dept_id=d.dept_id  
WHERE d.dept_id IS NULL
```

UNION

```
SELECT *  
FROM student s  
RIGHT JOIN department d  
ON s.dept_id=d.dept_id  
WHERE s.id IS NULL;
```

Q1 Department Wise Topper

</> SQL

```
SELECT s.name,  
       s.marks,  
       d.dept_name  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
WHERE s.marks=  
(  
  SELECT MAX(marks)  
  FROM student  
  WHERE dept_id=s.dept_id  
);
```


Q2 Department Average

SQL

```
SELECT d.dept_name,  
AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name;
```

Q3 Highest Average Department

SQL

```
SELECT d.dept_name,  
AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name  
ORDER BY AVG(s.marks) DESC  
LIMIT 1;
```

Q4 More than Overall Average

SQL

```
SELECT s.name,  
d.dept_name,  
s.marks  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
WHERE s.marks>  
(  
SELECT AVG(marks)  
FROM student  
);
```

Q5 Above Department Average

SQL

```
SELECT s.name,  
d.dept_name,  
s.marks  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
WHERE s.marks>  
(  
SELECT AVG(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

Q6 A Block Students

SQL

```
SELECT s.name,  
d.block_name  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
WHERE d.block_name='A-Block';
```

Q7 Highest Marks in Every Department

</> SQL

```
SELECT d.dept_name,  
MAX(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name;
```

Q8 Student Count

</> SQL

```
SELECT d.dept_name,  
COUNT(*)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name;
```

Q9 HOD Wise Average

SQL

```
SELECT d.hod_name,  
AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.hod_name;
```

Q10 Departments having Average >80

SQL

```
SELECT d.dept_name,  
AVG(s.marks)  
FROM student s  
JOIN department d  
ON s.dept_id=d.dept_id  
GROUP BY d.dept_name  
HAVING AVG(s.marks)>80;
```

1. UNION

Definition

Dono queries ka result combine karta hai aur **duplicate rows** hata deta hai.

Example

Students from Indore + Bhopal

 SQL

```
SELECT name, city
FROM student
WHERE city='Indore'

UNION

SELECT name, city
FROM student
WHERE city='Bhopal';
```

Result



n/c/6a512cab-dcd0-83ee-a37b-da062380811a

2. UNION ALL

Definition

Dono queries ko combine karta hai.

Duplicates remove nahi karta.

`</>` SQL

```
SELECT city
FROM student
WHERE city='Indore'
```

UNION ALL

```
SELECT city
FROM student
WHERE city='Indore';
```

Difference

UNION

Duplicate remove karta hai

Slow

DISTINCT jaisa behavior

UNION ALL

Duplicate remove nahi karta

Fast

Original data

7. Common Department IDs (Intersection)

SQL

```
SELECT dept_id
FROM department
WHERE dept_id IN
(
  SELECT dept_id
  FROM student
);
```


8. Departments having no students

`</>` SQL

```
SELECT dept_id
FROM department
WHERE dept_id NOT IN
(
  SELECT dept_id
  FROM student
);
```

9. Students from A-Block OR Marks > 90

SQL

```
SELECT name
FROM student
WHERE dept_id IN
(
SELECT dept_id
FROM department
WHERE block_name='A-Block'
)

UNION

SELECT name
FROM student
WHERE marks>90;
```

1. UNION

Definition

Dono queries ka result combine karta hai aur **duplicate rows hata deta hai**.

Example

Students from Indore + Bhopal

SQL

```
SELECT name, city
FROM student
WHERE city='Indore'

UNION

SELECT name, city
FROM student
WHERE city='Bhopal';
```

Result



n/c/6a512cab-dcd0-83ee-a37b-da062380811a

2. UNION ALL

Definition

Dono queries ko combine karta hai.

Duplicates remove nahi karta.

SQL

```
SELECT city
FROM student
WHERE city='Indore'

UNION ALL

SELECT city
FROM student
WHERE city='Indore';
```

Difference

UNION

Duplicate remove karta hai

Slow

DISTINCT jaisa behavior

UNION ALL

Duplicate remove nahi karta

Fast

Original data

8. Departments having no students

SQL

```
SELECT dept_id
FROM department
WHERE dept_id NOT IN
(
SELECT dept_id
FROM student
);
```

9. Students from A-Block OR Marks > 90

</> SQL

```
SELECT name
FROM student
WHERE dept_id IN
(
SELECT dept_id
FROM department
WHERE block_name='A-Block'
)

UNION

SELECT name
FROM student
WHERE marks>90;
```

10. Students in A-Block AND Marks > 90

`</>` SQL

```
SELECT *  
FROM student  
WHERE dept_id IN  
(  
  SELECT dept_id  
  FROM department  
  WHERE block_name='A-Block'  
)  
AND marks>90;
```

/c/6a512cab-dcd0-83ee-a37b-da062380811a

UNION

Students from IT

</> SQL

```
SELECT name
FROM student
WHERE dept_id=
(
SELECT dept_id
FROM department
WHERE dept_name='CSE'
)
```

UNION

```
SELECT name
FROM student
WHERE dept_id=
(
SELECT dept_id
FROM department
WHERE dept_name='IT'
```


INTERSECTION using JOIN

SQL

```
SELECT DISTINCT s.*  
FROM student s  
INNER JOIN department d  
ON s.dept_id=d.dept_id  
WHERE d.block_name='A-Block'  
AND s.marks>85;
```

Operator	Meaning
UNION	Combine + remove duplicates
UNION ALL	Combine + keep duplicates
INTERSECTION	Common records
EXCEPT	First – Second